

Priority Queue

Priority Queue (STL)

A **Priority Queue** is an **abstract data type** (ADT) that functions like a regular queue, but where each element has a "priority." The element with the highest priority is always served (or extracted) first.

- **Implementation:** In C++, the Standard Template Library (STL) `std::priority_queue` is typically implemented using a **Heap** data structure. By default, it's a **Max Heap**, meaning the largest element has the highest priority and is at the top.
- **Key Operations:**
 - `push()`: Inserts an element.
 - `top()`: Returns a reference to the element with the highest priority (the largest by default).
 - `pop()`: Removes the element with the highest priority.
 - `empty()`: Checks if the container is empty.
 - `size()`: Returns the number of elements.
- **Time Complexity:** All key operations (`push`, `pop`, `top`) take $O(\log n)$ time, where n is the number of elements.

C++ STL Example

```
#include <iostream>
#include <queue> // Header for std::priority_queue
#include <vector>
#include <functional> // Header for std::greater

void stl_priority_queue_example() {
    // By default, it's a **Max Heap** (largest element is on top)
    std::priority_queue<int> max_pq;

    max_pq.push(10);
    max_pq.push(50);
    max_pq.push(20);

    // Output: 50 (The largest element)
    std::cout << "Top element (Max Heap): " << max_pq.top() << std::endl;
    max_pq.pop(); // Removes 50

    // To create a **Min Heap** (smallest element on top):
    // Syntax: priority_queue<type, container, comparator>
    std::priority_queue<int, std::vector<int>, std::greater<int>> min_pq;
    min_pq.push(10);
    min_pq.push(50);
    min_pq.push(20);

    // Output: 10 (The smallest element)
```

```
std::cout << "Top element (Min Heap): " << min_pq.top() << std::end  
}
```

Heap

A **Heap** is a specialized **tree-based data structure** that satisfies the **Heap Property**. It is usually implemented as an array to save space and allows for efficient calculation of parent/child indices.

- **Structure:** Heaps are **complete binary trees**. A complete binary tree is one where all levels are fully filled, except possibly the last level, which is filled from left to right.
 - **Types:**
 - **Max Heap:** Satisfies the **Max Heap Property**.
 - **Min Heap:** Satisfies the **Min Heap Property**.
 - **Indexing (Array Implementation):** For a node at index i :
 - **Parent:** $\lfloor \frac{i-1}{2} \rfloor$
 - **Left Child:** $2i + 1$
 - **Right Child:** $2i + 2$
-

Heap Property

The **Heap Property** is the specific rule that defines the relationship between a parent node and its children in a heap.

1. Max Heap Property

For every node i (except the root), the value of the parent is **greater than or equal to** the value of the children. $\text{Parent}(i) \geq \text{Child}(i)$ This ensures that the **largest element is always at the root**.

2. Min Heap Property

For every node i (except the root), the value of the parent is **less than or equal to** the value of the children. $\text{Parent}(i) \leq \text{Child}(i)$ This ensures that the **smallest element is always at the root**.

Heapify (The Core Operation)

Heapify is the process of maintaining the **Heap Property** in a subtree (or the entire heap) starting from a given node. It ensures the tree structure remains a valid heap after an insertion or deletion.

- **Mechanism:** It works by comparing the node at index i with its children and swapping it with the **larger child (for Max Heap)** or **smaller child (for Min Heap)** if the property is violated. This process is then recursively called on the child's new position until the property is restored throughout the subtree.

- **Time Complexity:** $O(\log n)$, as the operation moves down one path in the tree.

C++ Code (Max Heapify)

```
// Function to maintain the Max Heap property in a subtree rooted at i
void maxHeapify(int arr[], int n, int i) {
    int largest = i;          // Initialize largest as root
    int left = 2 * i + 1;     // Left child index
    int right = 2 * i + 2;    // Right child index

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    // If right child is larger than current largest
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    // If largest is not the root
    if (largest != i) {
        std::swap(arr[i], arr[largest]); // Swap

        // Recursively heapify the affected sub-tree
        maxHeapify(arr, n, largest);
    }
}
```

Heap Construction (Building a Heap)

Heap Construction (or **Build Heap**) is the process of converting an arbitrary array into a valid heap.

- **Algorithm:** To build a heap of size n , you must run the **Heapify** operation on all non-leaf nodes, starting from the **last non-leaf node** and moving up to the root.
 - The last non-leaf node is at index $\lfloor \frac{n}{2} \rfloor - 1$ (assuming 0-based indexing).
- **Time Complexity:** $O(n)$

C++ Code (Building a Max Heap)

```
// Function to build a Max Heap from an array
void buildMaxHeap(int arr[], int n) {
    // Start from the last non-leaf node and move up to the root (index n/2 - 1)
    // The nodes from n/2 to n-1 are all leaf nodes.
    for (int i = n / 2 - 1; i >= 0; i--) {
        maxHeapify(arr, n, i);
    }
}
```